

Roman Marbel — Technical Requirements Document (TRD) v1.0

2026-09-19 · @Someone

1. Purpose & Scope

This is the **Technical Requirements Document (TRD)** for the Roman Marbel e-commerce and lead-capture platform. It translates the product decisions in the PRD (v1.0) into buildable engineering specifications: architecture, schema, APIs, algorithms, infrastructure, security and test strategy.

Relationship to the PRD. The PRD owns *what* and *why* — personas, goals, functional requirements, the quantity/pricing formulas, and the release phasing. This TRD owns *how* — the concrete stack, schema, contracts and operational detail an engineer needs to build and run it. Requirement IDs (F-x.x, N-x.x) from the PRD are referenced throughout rather than restated; where this document specifies something the PRD only described in prose (notably the calculation engine and the data model), this document is the source of truth for implementation.

In scope: the MVP (Phase 1) system in full technical detail, plus enough forward design in the schema and API surface that Phase 2 (quotes/orders/payments, multi-area estimates) does not require a rewrite.

Out of scope: UI visual design (owned by the roman-marbel-ui standard and Figma/design artifacts), and the Phase 3 AI features, which get their own technical spec once Phase 2 ships and the phase gate in PRD §14 is met.

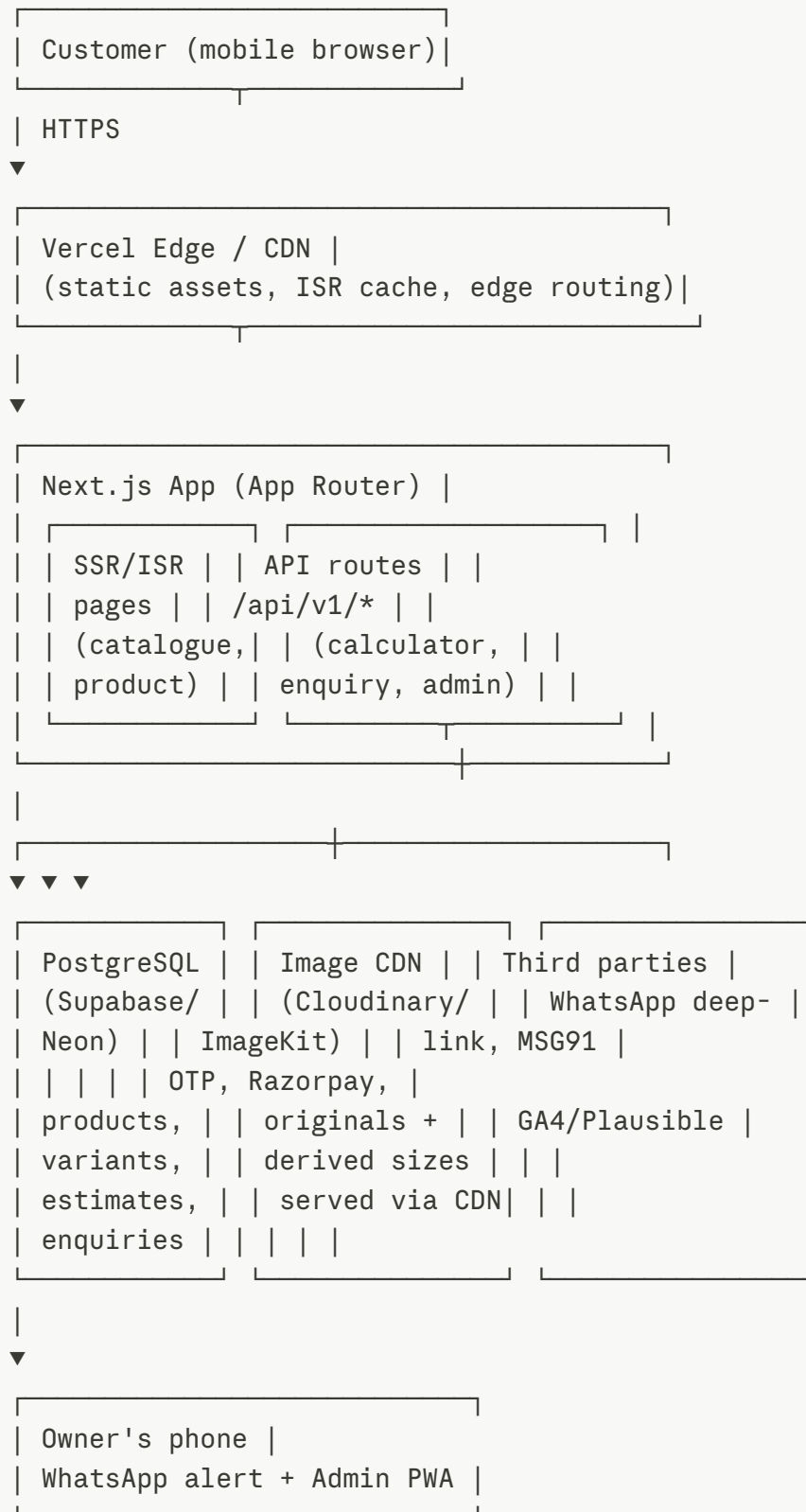
Audience: the engineer(s) building this, and the owner/CTO reviewing technical decisions and cost trade-offs.

2. System Architecture

2.1 Style

A **monolithic Next.js application** (single deployable, single repo) backed by a managed Postgres database, with heavy media handling delegated to a dedicated image CDN. No microservices, no message queue, no Kubernetes — justified in §3.3. The system is read-heavy (catalogue browsing) with bursty, low-volume writes (enquiries, admin edits).

2.2 Component diagram



2.3 Component responsibilities

Component	Responsibility	Notes
Next.js App Router pages	Catalogue, product, wizard, estimate — server-rendered or ISR for SEO and speed	See §3, §10
API routes (/api/v1/*)	Calculator, enquiry submission, OTP, admin CRUD, order status	Thin controllers over a service layer (§5)
Service layer	Business logic: pricing engine, zone resolution, lead scoring	Pure functions where possible — this is what gets unit-tested (§12)
PostgreSQL	System of record for catalogue, estimates, enquiries, orders	Managed, with PITR backup (§13)
Image CDN	Upload intake, transformation, global delivery	Never serve an unprocessed original to a customer (§7)
WhatsApp / SMS providers	Enquiry alerts, OTP delivery	Deep links in MVP; Business Cloud API in Phase 2 (§9)
Admin surface	Same Next.js app, /admin/* routes, role-gated	Not a separate app — one codebase, one deploy

2.4 Request-flow example: estimate → enquiry

1. Customer completes wizard or opens a product page
2. Client POSTs area/product to /api/v1/calculate
3. Service layer runs the pricing engine (§6), returns a breakdown
4. Client renders the Estimate screen; estimate persisted client-side (localStorage) with a reference no.
5. Customer submits enquiry → POST /api/v1/enquiries
6. Server: validates OTP, writes Enquiry + linked Estimate row, fires WhatsApp deep-link payload and SMS confirmation, returns 201
7. Admin sees the enquiry in /admin/enquiries within seconds (polling or a lightweight webhook, see §9.1)

2.5 Data flow boundaries

- **Catalogue reads never require the calculator or enquiry services** — the site must stay fully browsable even if the estimate/enquiry write path is degraded.

- **The calculator is a pure function of catalogue data + user input.** It has no side effects and can be exercised entirely by unit tests (§12) without a database.
- **Nothing customer-facing ever reads an image CDN original.** All customer-facing image URLs are CDN transformation URLs (§7).

3. Technology Stack

3.1 Core stack

Layer	Choice	Version (pin at project start)
Language	TypeScript	5.x, <code>strict: true</code>
Framework	Next.js, App Router	Latest stable LTS at kickoff
Runtime	Node.js	Latest Active LTS
Styling	Tailwind CSS	Latest stable
ORM	Prisma	Latest stable
Database	PostgreSQL	15+ (managed — Supabase or Neon)
Validation	Zod	For every API input and every form
Forms	React Hook Form	Wizard and enquiry forms
Image CDN	Cloudinary or ImageKit	Pick one at kickoff; do not build a custom pipeline
Auth	Custom phone-OTP (customer) + credential-based (admin), sessions via signed HTTP-only cookies	See §8
Payments	Razorpay	UPI-first, Phase 2
SMS/OTP	MSG91 or Fast2SMS	Indian delivery reliability over global providers (Twilio)
Hosting	Vercel	Frontend + API routes
Error tracking	Sentry	Free tier sufficient at this scale
Analytics	GA4 or Plausible	Plausible if cookie-consent simplicity matters more than free-tier GA4 features
Testing	Vitest (unit), Playwright (e2e)	See §12

Layer	Choice	Version (pin at project start)
CI/CD	GitHub Actions → Vercel	Preview deploy per PR

3.2 Why these choices, specifically

- **Next.js over a separate SPA + API.** One deployable, one auth story, ISR gives SEO without a separate SSR service. A Vite SPA would need its own SEO workaround (prerendering) and a separate backend to talk to.
- **Prisma over a raw query builder.** Type-safe queries matched to the TypeScript schema reduce the class of bug most likely in a small team: a field renamed in the DB but not in code. Migration files also double as schema documentation.
- **Zod at every boundary.** The calculator (§6) is the one place a silent bad input becomes a wrong price shown to a customer. Validate at the API boundary, not just in the UI.
- **Managed Postgres over self-hosted.** No one on this project should be tuning `postgresql.conf`. Automated backups and point-in-time recovery come out of the box (§13).
- **A single image CDN provider, chosen once.** Building custom resize/crop/format-negotiation logic is a multi-week project that a \$0–20/month CDN plan replaces entirely. This is the highest-leverage "don't build it yourself" decision in the whole stack.

3.3 Why not more, and why not less

Why not microservices / message queues: the entire write volume at 12-month target scale (§3.3 of the PRD: ~150 enquiries/month) is roughly one write every few hours. A queue, a service mesh, or per-domain services add operational surface with zero throughput benefit at this scale, and every added service is something the eventual 0.2 FTE maintainer has to understand.

Why not a static site generator alone (e.g. Astro) with no backend: the calculator, OTP, enquiry capture and admin CRUD are inherently dynamic and stateful. A pure static site would need a separate backend anyway — Next.js gives both in one project.

Why not a low-code platform (Bubble, Glide, etc.): the pricing engine (§6) has enough branching logic (pricing unit, wastage tiers, zone-based delivery) that it will fight a low-code data-flow model within a few months, and the owner's requirement for a fast, lightweight mobile experience (PRD §9.1) is hard to guarantee on most low-code hosts.

4. Database Schema

PostgreSQL, managed via Prisma migrations. All primary keys `uuid` (`gen_random_uuid()`). All tables carry `created_at timestamptz default now()` and `updated_at timestamptz` (auto-updated via trigger or Prisma `@updatedAt`). Nothing is hard-deleted — every table has `deleted_at timestamptz null`, and all queries filter `WHERE deleted_at IS NULL` by default.

```
CREATE TYPE pricing_unit AS ENUM ('box', 'sqft', 'slab');
CREATE TYPE stock_status AS ENUM ('in', 'low', 'out', 'on_order');
CREATE TYPE budget_band AS ENUM ('economy', 'value', 'premium');
CREATE TYPE enquiry_type AS ENUM ('standard', 'bulk', 'sample', 'site_visit', 'restock');
CREATE TYPE enquiry_status AS ENUM ('new', 'contacted', 'quoted', 'visited', 'won', 'lost');
CREATE TYPE order_status AS ENUM ('confirmed', 'packed', 'out_for_delivery', 'delivered', 'cancelled');
CREATE TYPE payment_status AS ENUM ('unpaid', 'advance_paid', 'paid_in_full');
CREATE TYPE customer_type AS ENUM ('retail', 'contractor', 'builder');

CREATE TABLE category (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name_en text NOT NULL,
  name_bn text,
  slug text UNIQUE NOT NULL,
  parent_id uuid REFERENCES category(id),
  sort_order int DEFAULT 0,
  icon text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now(),
  deleted_at timestamptz
);

CREATE TABLE product (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  sku text UNIQUE,
  name_en text NOT NULL,
  name_bn text,
  slug text UNIQUE NOT NULL,
  category_id uuid NOT NULL REFERENCES category(id),
  material text NOT NULL, -- marble | granite | vitrified | ceramic | other
  size_mm text, -- '600x600'
  finish text, -- default finish; variants override, see product_variant
  colour_family text[] NOT NULL DEFAULT '{}',
  colour_hex_primary text, -- auto-extracted, see §7
```

```

pricing_unit pricing_unit NOT NULL DEFAULT 'box',
rate_per_sqft numeric(10,2) NOT NULL,
mrp_per_sqft numeric(10,2),
pieces_per_box int,
sqft_per_box numeric(6,2),
weight_per_sqft numeric(6,3),
stock_status stock_status NOT NULL DEFAULT 'in',
budget_band budget_band NOT NULL DEFAULT 'value',
application text[] NOT NULL DEFAULT '{}', --
floor|wall|bathroom|kitchen|outdoor|stairs|parking
is_published boolean NOT NULL DEFAULT false,
is_featured boolean NOT NULL DEFAULT false,
shade_variation_note text,
rate_updated_at timestampz NOT NULL DEFAULT now(),
created_at timestampz DEFAULT now(),
updated_at timestampz DEFAULT now(),
deleted_at timestampz
);
CREATE INDEX idx_product_category ON product(category_id) WHERE deleted_at IS
NULL;
CREATE INDEX idx_product_published ON product(is_published) WHERE deleted_at
IS NULL;
CREATE INDEX idx_product_colour ON product USING GIN(colour_family);
CREATE INDEX idx_product_application ON product USING GIN(application);

CREATE TABLE product_variant ( -- added after UI concept review, PRD §6.8/§13
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
product_id uuid NOT NULL REFERENCES product(id),
thickness_mm int,
finish text NOT NULL, -- polished|honed|leather|matt|glossy|anti_skid
retail_rate_per_sqft numeric(10,2) NOT NULL,
dealer_rate_per_sqft numeric(10,2),
stock_status stock_status NOT NULL DEFAULT 'in',
lot_note text,
water_absorption_pct numeric(5,2),
compressive_strength_mpa numeric(6,1),
quarry_origin text,
created_at timestampz DEFAULT now(),
updated_at timestampz DEFAULT now(),
deleted_at timestampz,
UNIQUE(product_id, thickness_mm, finish)
);

CREATE TABLE product_image (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
product_id uuid NOT NULL REFERENCES product(id),

```

```

cdn_public_id text NOT NULL,
type text NOT NULL, -- flat|context|scale
sort_order int DEFAULT 0,
alt_en text,
alt_bn text,
created_at timestamptz DEFAULT now()
);

CREATE TABLE zone (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  pin_codes text[] NOT NULL DEFAULT '{}',
  delivery_flat_charge numeric(10,2) DEFAULT 0,
  free_above numeric(12,2),
  min_order numeric(12,2),
  max_auto_weight_kg numeric(10,2),
  site_visit_available boolean DEFAULT false,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);
CREATE INDEX idx_zone_pins ON zone USING GIN(pin_codes);

CREATE TABLE customer (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  phone text UNIQUE NOT NULL,
  phone_verified boolean DEFAULT false,
  name text,
  pin_code text,
  type customer_type NOT NULL DEFAULT 'retail',
  trade_discount_pct numeric(5,2) DEFAULT 0,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);

CREATE TABLE estimate ( -- anonymous-first; customer_id nullable
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  reference_no text UNIQUE NOT NULL, -- RM-2609-0147
  session_id text NOT NULL,
  customer_id uuid REFERENCES customer(id),
  pin_code text,
  zone_id uuid REFERENCES zone(id),
  wastage_pct numeric(5,2) NOT NULL DEFAULT 8.00,
  subtotal numeric(12,2) NOT NULL,
  addons_total numeric(12,2) NOT NULL DEFAULT 0,
  delivery_charge numeric(10,2) NOT NULL DEFAULT 0,
  gst_amount numeric(12,2) NOT NULL DEFAULT 0,

```

```

total numeric(12,2) NOT NULL,
source text NOT NULL, -- browse|wizard
created_at timestamptz DEFAULT now(),
expires_at timestamptz NOT NULL DEFAULT (now() + interval '30 days')
);

CREATE TABLE estimate_line (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  estimate_id uuid NOT NULL REFERENCES estimate(id) ON DELETE CASCADE,
  product_id uuid NOT NULL REFERENCES product(id),
  variant_id uuid REFERENCES product_variant(id),
  area_label text, -- 'Living room'
  length_ft numeric(8,2),
  width_ft numeric(8,2),
  area_sqft numeric(10,2) NOT NULL,
  area_with_wastage numeric(10,2) NOT NULL,
  boxes int,
  actual_sqft numeric(10,2) NOT NULL,
  rate_applied numeric(10,2) NOT NULL, -- snapshot, never recomputed from
product
  line_total numeric(12,2) NOT NULL,
  skirting_ft numeric(8,2),
  adhesive_bags int,
  grout_kg numeric(6,2)
);

CREATE TABLE enquiry (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  estimate_id uuid REFERENCES estimate(id),
  name text NOT NULL,
  phone text NOT NULL,
  phone_verified boolean DEFAULT false,
  pin_code text,
  zone_id uuid REFERENCES zone(id),
  preferred_contact_time text,
  message text,
  type enquiry_type NOT NULL DEFAULT 'standard',
  status enquiry_status NOT NULL DEFAULT 'new',
  lost_reason text,
  assigned_to uuid, -- references admin_user(id), see §8
  lead_score int,
  source_page text,
  created_at timestamptz DEFAULT now(),
  last_contacted_at timestamptz
);
CREATE INDEX idx_enquiry_status ON enquiry(status);

```

```
CREATE INDEX idx_enquiry_created ON enquiry(created_at DESC);
```

```
CREATE TABLE quote ( -- Phase 2
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  enquiry_id uuid NOT NULL REFERENCES enquiry(id),
  quoted_total numeric(12,2) NOT NULL,
  valid_until date NOT NULL,
  pdf_cdn_url text,
  created_at timestamptz DEFAULT now()
);
```

```
CREATE TABLE "order" ( -- Phase 2, quoted table name to avoid the SQL keyword
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  enquiry_id uuid REFERENCES enquiry(id),
  quote_id uuid REFERENCES quote(id),
  customer_id uuid NOT NULL REFERENCES customer(id),
  order_code text UNIQUE NOT NULL, -- RM-2409-118
  quoted_total numeric(12,2),
  final_total numeric(12,2) NOT NULL,
  advance_paid numeric(12,2) DEFAULT 0,
  payment_status payment_status NOT NULL DEFAULT 'unpaid',
  delivery_status order_status NOT NULL DEFAULT 'confirmed',
  delivery_address text,
  delivery_date date,
  vehicle_note text,
  razorpay_payment_id text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);
```

```
CREATE TABLE order_status_event ( -- powers the F-9.3 step tracker
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id uuid NOT NULL REFERENCES "order"(id) ON DELETE CASCADE,
  status order_status NOT NULL,
  note text,
  occurred_at timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE TABLE admin_user (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  phone text UNIQUE NOT NULL,
  name text NOT NULL,
  password_hash text NOT NULL,
  role text NOT NULL DEFAULT 'owner', -- owner|staff
  is_active boolean DEFAULT true,
  created_at timestamptz DEFAULT now()
);
```

```
);

CREATE TABLE setting ( -- every business constant lives here, never in code
  key text PRIMARY KEY,
  value jsonb NOT NULL,
  updated_at timestamptz DEFAULT now(),
  updated_by uuid REFERENCES admin_user(id)
);
-- seed rows: gst_rate_pct, default_wastage_pct,
adhesive_coverage_sqft_per_bag,
-- grout_factor_kg_per_sqft, rate_validity_text
```

4.1 Notes on the schema

- `estimate_line.rate_applied` is a **snapshot**, deliberately denormalised. When `product.rate_per_sqft` changes, historical estimates must still show the number the customer actually saw — this directly implements PRD §13's design note.
- `product_variant` is optional per product — a simple product with no variants can be priced straight off `product.rate_per_sqft`; variant rows only exist where thickness/finish actually branches pricing or stock.
- `order_status_event` is an append-only log rather than a single mutable status column on `order`, so the F-9.3 step tracker can show a timestamp per stage without a separate audit table.
- No table is ever `DELETE` d from in application code; `deleted_at` plus a scheduled hard-purge job (12+ months, GDPR/DPDP-style retention policy, §13) is the only path to real deletion.

5. API Specification

REST, JSON, versioned under `/api/v1/`. All responses: `{ data, error }`. Errors use a consistent shape: `{ error: { code, message, fields? } }`. Public endpoints are cached at the CDN edge where marked; write endpoints are never cached.

5.1 Catalogue (public, read-heavy — cache aggressively)

Method & path	Purpose	Cache
GET <code>/api/v1/categories</code>	List categories, tree structure	ISR, 1 hour

Method & path	Purpose	Cache
GET /api/v1/products	List/filter/sort products. Query: category, colour, material, size, finish, budget_band, room, in_stock_only, sort, page, page_size	ISR, 15 min
GET /api/v1/products/:slug	Full product detail incl. variants and images	ISR, 15 min
GET /api/v1/products/:slug/similar	Related products (Phase 2, embedding-based)	ISR, 1 hour
GET /api/v1/search?q=	Fuzzy + Banglish-tolerant text search	No cache (query-dependent)
GET /api/v1/zones/lookup?pin=	Resolve a PIN to zone, delivery terms, site-visit availability	ISR, 1 hour

5.2 Calculator (stateless, pure function)

Method & path	Purpose
POST /api/v1/calculate	Body: { items: [{ product_id, variant_id?, length_ft, width_ft, wastage_pct? }], pin_code, addons?: {skirting, adhesive, grout} }. Returns the full breakdown per PRD §7: area, wastage, boxes, rate, subtotal, addons, delivery, GST, total. No database write.

This endpoint must be implemented as a thin wrapper over the pure calculation engine (§6) — no side effects, fully unit-testable without a running server.

5.3 Estimates & enquiries

Method & path	Purpose
POST /api/v1/estimates	Persists a calculated estimate (from §5.2's output), returns reference_no
GET /api/v1/estimates/:reference_no	Retrieve a saved estimate (for share links / "resume later")

Method & path	Purpose
POST /api/v1/otp/request	Body: { phone }. Rate-limited (§5.5)
POST /api/v1/otp/verify	Body: { phone, code }. Returns a short-lived verified-phone token
POST /api/v1/enquiries	Body: { estimate_id?, name, phone, verified_phone_token, pin_code, preferred_contact_time, message?, type }. Triggers WhatsApp + SMS (§9). Returns enquiry id
GET /api/v1/orders? phone=&otp_token=	Phone-based lookup for the customer order tracker (F-9.1), OTP-gated, no full login required
GET /api/v1/orders/:order_code	Single order detail with status timeline

5.4 Admin (/api/v1/admin/* , all require session auth, see §8)

Method & path	Purpose
POST /api/v1/admin/login	Phone + password → session cookie
GET/POST /api/v1/admin/products	List / create products
PATCH /api/v1/admin/products/:id	Partial update — must support single-field patch (e.g. { rate_per_sqft: 55 }) so the "tap the number, type, done" admin flow (PRD A-1.3) is one call, not a full-object PUT
POST /api/v1/admin/products/:id/images	Upload → proxies to image CDN, stores cdn_public_id
PATCH /api/v1/admin/products/:id/stock	Toggle stock status
PATCH /api/v1/admin/products/bulk-rate	Body: { category_id, pct_change } — A-1.6
GET/POST /api/v1/admin/variants	Manage product_variant rows
GET /api/v1/admin/enquiries	List with filters: status, zone, date range
PATCH /api/v1/admin/enquiries/:id	Update status, notes, assignment
POST /api/v1/admin/quotes	Create a quote from an enquiry (Phase 2)
POST /api/v1/admin/orders	Create an order from a quote (Phase 2)

Method & path	Purpose
PATCH <code>/api/v1/admin/orders/:id/status</code>	Append an <code>order_status_event</code> row
GET <code>/api/v1/admin/reports/weekly</code>	The §8.6 (PRD) reporting screen data

5.5 Cross-cutting API rules

Rule	Detail
Rate limiting	POST <code>/otp/request</code> : 3/hour per phone, 10/hour per IP. POST <code>/enquiries</code> : 5/hour per IP. POST <code>/calculate</code> : 60/min per IP (cheap, but still bot-protected)
Idempotency	POST <code>/enquiries</code> and POST <code>/estimates</code> accept an <code>Idempotency-Key</code> header to survive flaky mobile networks retrying a submit
Input validation	Every body validated against a Zod schema before touching the service layer; invalid input returns 400 with per-field messages, never a 500
Auth errors	401 unauthenticated, 403 authenticated but wrong role — admin endpoints never leak whether a resource exists to an unauthorised caller (404 , not 403 , for cross-tenant-style access — though at single-shop scale this mainly matters for <code>assigned_to</code> staff scoping)
Pagination	Cursor-based on <code>products</code> and <code>enquiries</code> list endpoints once catalogue exceeds ~500 items; offset-based is fine below that and simpler to ship first

6. Calculation Engine

This implements PRD §7. It is the single most business-critical piece of code in the system — see PRD's counter-metric on estimate variance. It must be a **pure function**: same input always produces the same output, no I/O, no `Date.now()` inside the core math (pass `now` in if needed for GST-rate-effective-date logic).

6.1 Contract

```
type PricingUnit = 'box' | 'sqft' | 'slab';

interface AreaInput {
  areaLabel?: string;
  lengthFt: number;
  widthFt: number;
}

interface CalcItemInput {
  productId: string;
  variantId?: string;
  areas: AreaInput[]; // supports "add another area" for the same product
  wastagePctOverride?: number; // user-adjustable, PRD F-5.3, default from
settings
  includeSkirting?: boolean;
  includeAdhesive?: boolean;
  includeGrout?: boolean;
}

interface CalcRequest {
  items: CalcItemInput[];
  pinCode: string;
}

interface CalcLineResult {
  productId: string;
  variantId?: string;
  areaSqft: number;
  wastagePct: number;
  areaWithWastage: number;
  pricingUnit: PricingUnit;
  boxes: number | null; // null for slab pricing
  actualSqftPurchased: number;
  ratePerSqft: number;
  materialCost: number;
  skirtingCost: number;
  adhesiveCost: number;
  adhesiveBags: number;
  groutCost: number;
  groutKg: number;
  needsShopConfirmation: boolean; // true for slab material, or weight above
auto-delivery threshold
}
```

```

interface CalcResult {
  lines: CalcLineResult[];
  materialSubtotal: number;
  addonsTotal: number;
  zone: { id: string; name: string } | null;
  deliveryCharge: number;
  deliveryNeedsQuote: boolean;
  gstAmount: number;
  total: number;
  warnings: string[]; // e.g. "PIN not in service area – delivery to be confirmed"
}

function calculate(req: CalcRequest, catalogue: CatalogueSnapshot, settings: Settings): CalcResult;

```

6.2 Algorithm (per PRD §7.1–§7.8)

```

for each item in request.items:
  product ← catalogue.getProduct(item.productId)
  variant ← item.variantId ? catalogue.getVariant(item.variantId) : null
  rate ← variant?.retailRate ?? product.ratePerSqft // role resolution happens
  // one level up, in the API handler – see §6.4

  areaSqft ← sum(a.lengthFt * a.widthFt for a in item.areas)
  wastagePct ← item.wastagePctOverride ?? defaultWastageFor(product) // §7.2
  table
  areaWithWastage ← areaSqft * (1 + wastagePct / 100)

  if product.pricingUnit == 'box':
    boxes ← ceil(areaWithWastage / product.sqftPerBox)
    actualSqft ← boxes * product.sqftPerBox
  else if product.pricingUnit == 'sqft' or 'slab':
    boxes ← null
    actualSqft ← areaWithWastage
  needsShopConfirmation ← true // slabs are irregular, PRD §7.3

  materialCost ← actualSqft * rate

  if item.includeSkirting:
    perimeter ← sum(2 * (a.lengthFt + a.widthFt) for a in item.areas)
    skirtingCost ← perimeter * settings.skirtingRatePerFt
  if item.includeAdhesive:
    adhesiveBags ← ceil(areaWithWastage / settings.adhesiveCoverageSqftPerBag)
    adhesiveCost ← adhesiveBags * settings.adhesiveBagPrice
  if item.includeGrout:

```

```

groutKg ← areaWithWastage * settings.groutFactorKgPerSqft
groutCost ← groutKg * settings.groutPricePerKg

emit CalcLineResult

materialSubtotal ← sum(line.materialCost for line in lines)
addonsTotal ← sum(line.skirtingCost + line.adhesiveCost + line.groutCost for
line in lines)

zone ← resolveZone(request.pinCode) // §6.3
totalWeightKg ← sum(line.actualSqft * product.weightPerSqft for line in
lines)

if zone is null:
  deliveryCharge ← 0; deliveryNeedsQuote ← true
  warnings.push("PIN not in service area – delivery to be confirmed")
else if totalWeightKg > zone.maxAutoWeightKg:
  deliveryCharge ← 0; deliveryNeedsQuote ← true // PRD §7.6, "large order –
quoted"
else:
  baseCharge ← (materialSubtotal + addonsTotal) >= zone.freeAbove ? 0 :
zone.deliveryFlatCharge
  deliveryCharge ← baseCharge; deliveryNeedsQuote ← false

gstAmount ← (materialSubtotal + addonsTotal) * settings.gstRatePct / 100
total ← materialSubtotal + addonsTotal + deliveryCharge + gstAmount

return CalcResult{ lines, materialSubtotal, addonsTotal, zone,
deliveryCharge,
deliveryNeedsQuote, gstAmount, total, warnings }

```

6.3 Zone resolution

```

function resolveZone(pinCode, zones):
  return zones.find(z => z.pinCodes.includes(pinCode)) ?? null

```

PIN-to-zone is a flat lookup table (`zone.pin_codes[]`), maintained by the owner via admin, not hardcoded — PIN boundaries and zone assignments will be revised as delivery capacity changes (see PRD §2.2's open question on the 713401/713407 discrepancy).

6.4 Rate resolution (role-aware, keeps §6.2 pure)

Role resolution is deliberately **not** inside the pure `calculate()` function — it is resolved once, before the call, by the API handler:

```
function resolveRate(product, variant, customer):
  if customer?.type in ['contractor', 'builder'] and variant?.dealerRate
  exists:
    return variant.dealerRate
  return variant?.retailRate ?? product.ratePerSqft
```

This keeps the core engine free of auth/session concerns and trivially testable, while still implementing PRD F-8.6 (account-based rate resolution, never a client-toggable preview).

6.5 Edge cases the implementation and its tests must cover

Case	Required behaviour
Zero or negative dimension	<code>400 VALIDATION_ERROR</code> , never silently coerced to 0
Absurd input (e.g. 9,999 ft)	Accepted but flagged: <code>warnings.push("Unusually large area – please confirm with the shop")</code> above a configurable threshold
Wastage below 0% or above 25%	Clamped server-side to <code>[settings.minWastagePct, settings.maxWastagePct]</code> regardless of client input
<code>sqftPerBox</code> is 0 or null on a box-priced product	Treated as a data error: <code>500</code> in admin context (bad catalogue data), never a divide-by-zero surfaced to a customer
Multiple areas, same product	Areas sum before wastage is applied once to the total, not per-area, unless each area explicitly requests its own wastage override
PIN outside all zones	<code>zone: null</code> , <code>deliveryNeedsQuote: true</code> , warning shown — never a hard error that blocks the estimate
Rate changed between page load and submit	<code>estimate_line.rate_applied</code> always uses the rate fetched at calculation time; a stale client-cached rate is revalidated server-side in <code>POST /calculate</code> , never trusted from the client

6.6 Regression suite (mandatory before launch)

The PRD §7.9 worked example (14×12 ft room, 600×600 vitrified at ₹52/sq ft, Zone A → ₹15,944 total) is codified as `calculate.worked-example.test.ts`. Any change to §6.2 that moves that number must justify the change in the PR description. This test, plus the edge-case table above as individual test cases, form the minimum bar for merging any change to this module.

7. Image Pipeline

This is the single highest-leverage piece of engineering for PRD §9.1's performance budget (LCP < 2.5s on 4G). Implements PRD §11.4.

7.1 Upload path (admin, from a phone)

1. Owner picks/takes a photo in the admin PWA
2. Client-side: downscale to max 2000px longest edge, re-encode as JPEG q~85, before upload (saves upload time on rural 4G — a 12MP phone photo can be 8-12MB; downscaled it's under 1MB)
3. Upload directly to image CDN (signed upload, client → CDN, not proxied through our server — avoids doubling bandwidth cost and server load)
4. CDN returns a `public_id`; `POST /api/v1/admin/products/:id/images` stores it, plus type (`flat|context|scale`) and `sort_order`
5. Server-side: run colour extraction (§7.3) against the CDN-hosted image, store `colour_hex_primary` and update `colour_family` on the product

7.2 Delivery path (customer-facing)

Context	Transformation	Target size
Grid thumbnail	<code>w_400,h_500,c_fill,q_auto,f_auto</code>	≤ 40 KB
Product page hero	<code>w_800,h_1000,c_fill,q_auto,f_auto</code>	≤ 180 KB
Full-screen zoom	<code>w_1600,q_auto,f_auto</code>	≤ 400 KB
WhatsApp share preview	<code>w_1200,h_630,c_fill,q_auto,f_auto</code> (OG image ratio)	—

`f_auto` lets the CDN negotiate AVIF/WebP/JPEG by browser support; `q_auto` picks quality per-image rather than a fixed number. **No original-resolution image URL is ever rendered in customer-facing HTML** — this is a hard rule, not a default to be overridden per-component.

7.3 Colour auto-tagging (PRD §12.2, Tier 1 AI)

```
on image upload (server-side job, async, does not block the admin save):
  fetch the CDN-hosted flat/close-up image
  run k-means clustering (k=5) on pixel RGB values, ignoring near-white/near-black
  outliers that are usually background, not the material
  map dominant cluster centroid to nearest entry in a fixed colour_family palette
  (white, beige, grey, brown, black, wood, multi)
  write colour_hex_primary and append to product.colour_family if not already present
```

Runs as a lightweight background job (a Vercel cron / queued function), not inline in the upload request — the admin save must never wait on this.

7.4 Consistency requirement

Every product needs the same three shot types (PRD F-1.3: flat/close-up, laid-floor context, scale/edge) under the same lighting and white balance. This is a photography-process requirement, not an engineering one, but the admin upload flow should **enforce the three-slot structure** (three clearly labelled upload slots per product, not a generic gallery) so an incomplete or inconsistent product is visibly incomplete in the admin UI before publish.

8. Authentication & Authorization

8.1 Customer identity: phone OTP

Customers never create a password. Phone number is the identity, verified via OTP, used for: enquiry submission, order lookup (F-9.1), and Phase 2 contractor rate resolution.

```
POST /api/v1/otp/request { phone }
→ generate 6-digit code, store hashed with a 5-min TTL in a short-lived
table
(otp_challenge: phone, code_hash, expires_at, attempt_count)
→ send via SMS provider (§9.2)
→ rate-limited: 3 requests/hour per phone, 10/hour per IP (PRD N-4.4)

POST /api/v1/otp/verify { phone, code }
→ check against otp_challenge, max 5 attempts before logout
→ on success: issue a short-lived verified-phone JWT (15 min), scoped to
that phone
only – this token is what POST /enquiries and GET /orders require, not a
full
session, since most customers verify once and complete one action
```

No customer session/cookie is maintained beyond this narrow, purpose-scoped token. This deliberately avoids building a full customer-account system for MVP (PRD's non-goal: no forced account creation).

8.2 Admin identity

```
POST /api/v1/admin/login { phone, password }
→ bcrypt/argon2 verify against admin_user.password_hash
→ on success: signed, HTTP-only, Secure, SameSite=Lax session cookie, 7-day
expiry
→ session carries { admin_user_id, role }
```

No self-service admin signup. Admin accounts are created by seed script / direct DB insert at project setup — there is exactly one owner account at launch, staff accounts added in Phase 2.

8.3 Roles

Role	Can	Cannot
owner	Everything under <code>/admin/</code> : catalogue, rates, enquiries, quotes, orders, settings, reports	—
staff (Phase 2)	View/update enquiries assigned to them, view catalogue, mark orders' status	Edit rates, publish/unpublish products, change global settings

Enforced via middleware on every `/api/v1/admin/*` route reading `role` off the session; never trust a role claim from the request body.

8.4 What this explicitly does not include at MVP

- No OAuth/social login — irrelevant to this user base.
- No customer password-based accounts — OTP-only is both simpler to build and matches how this audience actually verifies identity (they already trust OTP from banking/UPI apps).
- No API keys for third parties — this is not a platform with external developers in Phase 1/2.

8.5 Session & token security

Control	Detail
Cookies	<code>HttpOnly</code> , <code>Secure</code> , <code>SameSite=Lax</code>
CSRF	Same-site cookie policy plus an origin check on state-changing admin routes
Password storage	bcrypt (cost ≥ 12) or argon2id, never reversible encryption
OTP codes	Never logged in plaintext; hashed at rest, same as passwords
Token scope	The verified-phone JWT is scoped to exactly the phone verified — cannot be replayed for a different phone's enquiry or order lookup

9. Third-Party Integrations

9.1 WhatsApp

Phase	Mechanism	How it works
MVP	wa.me deep links	On enquiry submit, server constructs <code>https://wa.me/<owner_number>?text=<url-encoded summary></code> and returns it to the client, which opens it in a new tab/app. Zero cost, zero API approval needed, works day one. The owner receives a normal WhatsApp message he can reply to directly.
Phase 2	WhatsApp Business Cloud API	Enables: delivery receipts (know the owner actually saw it), templated order-status pushes, and a shared inbox if staff are added. Requires Meta Business verification — budget 1–2 weeks lead time before Phase 2 starts.

MVP limitation to design around: a wa.me link only pre-fills a message; it does not confirm delivery or reply programmatically. The admin enquiry inbox is therefore the actual source of truth for "did this reach the owner", not the WhatsApp send itself — the enquiry row is written to the database *before* the WhatsApp link fires, never dependent on it.

9.2 SMS / OTP

MSG91 or Fast2SMS — chosen for Indian DLT (Distributed Ledger Technology) template registration support, required by TRAI for transactional SMS in India. Twilio is avoided as primary here: DLT registration friction and per-SMS cost for Indian routes are both worse.

DLT compliance note: every SMS template (OTP, order confirmation) must be pre-registered with the telecom regulator via the provider's DLT portal before it can send. This registration takes 1–3 business days — start it in Phase 0, not Phase 1 week 6.

9.3 Payments — Razorpay (Phase 2)

Use	Detail
Advance/token collection	UPI-first checkout, ₹500–₹5,000 range (PRD F-6.7)
Full payment	Phase 2/3, part-payment schedules

Use	Detail
Webhook	<code>payment.captured</code> webhook updates <code>order.payment_status</code> and <code>order.razorpay_payment_id</code> — never mark an order paid purely from client-side redirect success, always confirm via webhook or server-side payment verification
PCI scope	None — Razorpay Checkout is hosted/embedded; card and UPI details never touch our servers

9.4 Image CDN

Cloudinary or ImageKit (pick one at kickoff — do not integrate both). Signed direct-upload from the admin client, transformation URLs for all delivery (§7). Both providers have generous free tiers that comfortably cover this project's volume through Phase 2.

9.5 Analytics

GA4 or Plausible. Custom events required regardless of provider: `product_view`, `filter_applied`, `wizard_step` (with step number), `wizard_complete`, `estimate_generated`, `whatsapp_click`, `enquiry_submit`, `order_lookup`. These feed the funnel visibility the PRD's §9.6 calls for and the "viewed but never enquired" report in PRD §8.6.

9.6 Maps

Google Maps embed (static or lightweight JS embed, not the full JS API) on the store/contact page for directions — a single embedded map, no interactive map SDK needed elsewhere in the product.

9.7 Integration failure handling

Integration	If it's down	Behaviour
Image CDN	Down	Serve last-known-good cached transformation URLs at the edge; never block page render waiting on the CDN
SMS/OTP provider	Down	OTP request fails gracefully with "couldn't send code, try again" — never silently accept an unverified phone

Integration	If it's down	Behaviour
WhatsApp deep link	User has no WhatsApp	Fall back to a plain "Call us" button, always shown alongside the WhatsApp button, never WhatsApp-only
Razorpay	Down	Enquiry/order flow (unpaid) still works — payment is additive, never a blocker to submitting an enquiry

10. Infrastructure & Deployment

10.1 Environments

Environment	Purpose	Data
local	Developer machines	Seeded fixture data, mirrors production schema
staging	Owner review before anything goes live	A real-ish copy of the catalogue; never real customer PII
production	Live site	Real data, real backups, real monitoring

10.2 Hosting topology

Vercel (production + staging projects, separate env vars)

- └ Next.js app (frontend + API routes) – auto-scaled, edge-cached static assets

- └ Vercel Cron – scheduled jobs (colour-tag backfill, estimate expiry cleanup,

- | nightly report pre-computation)

- └ Environment variables – DB URL, CDN keys, SMS/Razorpay keys, JWT secret, split per environment, never committed to the repo

Managed Postgres (Supabase or Neon)

- └ production instance – automated daily backups + PITR

- └ staging instance – separate, smaller tier

Image CDN (Cloudinary/ImageKit) – one account, folder-scoped by environment

10.3 CI/CD

On every PR:

1. Lint + typecheck (tsc --noEmit)
2. Unit tests (Vitest) – including the §6.6 calculator regression suite; a failing calculator test blocks merge, no exceptions
3. Build
4. Vercel preview deploy – a real URL the owner or reviewer can open on a phone

On merge to main:

5. Deploy to staging automatically
6. Deploy to production: manual promote (one click), never automatic on merge –
a human confirms staging looks right first, given the owner reviews on his own phone before anything customer-facing changes

On a schedule:

7. Playwright e2e smoke suite against staging nightly (§12.3)
8. Dependency update PRs (Dependabot/Renovate), reviewed weekly

10.4 Scaling posture

At the traffic level in PRD §2.4 (thousands of sessions/month at maturity), Vercel's hobby-to-pro tier and a managed Postgres small instance handle this with headroom. **No horizontal scaling design is needed for MVP or Phase 2.** If organic growth ever pushes past ~50,000 sessions/month, the first move is read replicas and a CDN cache-control audit, not a re-architecture — the monolith doesn't need to change shape for at least 10x current target traffic.

10.5 Configuration management

All business constants (GST rate, wastage defaults, adhesive coverage, delivery thresholds) live in the `setting` table (§4), editable from admin — **never** as hardcoded values or environment variables requiring a redeploy. Environment variables are reserved for infrastructure secrets (API keys, DB URL, JWT secret) that genuinely differ per environment and must never be owner-editable.

11. Observability

11.1 Error tracking

Sentry, wired from day one, both client and server. Two alert tiers:

Severity	Trigger	Response
Critical — page someone immediately	Error in <code>POST /api/v1/calculate</code> or <code>POST /api/v1/enquiries</code> ; any 5xx on those two routes	These are the two routes where a silent failure directly costs a lead or shows a wrong price. Alert via SMS/call, not just email.
Standard	Any other unhandled error	Daily digest, reviewed same day

11.2 Logging

Structured JSON logs (not free-text) for every API request: `{ timestamp, route, method, status, durationMs, requestId }`. Request ID generated at the edge and threaded through to Sentry so a customer-reported issue ("the calculator gave me a weird number") can be traced to the exact request and its inputs.

Never log: phone numbers in plaintext, OTP codes, password hashes, full addresses — log a truncated/hashed identifier where correlation is needed (PRD N-4.3, N-4.6).

11.3 Dashboards

Dashboard	Metrics	Audience
Product health	LCP/CLS (real-user via Vercel Analytics or a lightweight RUM), API p50/p95 latency, error rate	Engineer
Business funnel	product_view → estimate_generated → enquiry_submit, by day	Owner + engineer, matches PRD §3.3 and §9.6
Ops	Enquiry response time, catalogue freshness (% products rate-updated in 60 days)	Owner, weekly review per PRD §8.6

11.4 Alerting thresholds

Signal	Threshold	Action
Calculator error rate	> 1% of requests in 15 min	Page
Enquiry submit failure rate	> 5% of attempts in 15 min	Page
LCP p75	> 3.5s over 1 hour	Standard alert — investigate within the day
Database CPU/connections	> 80% sustained 10 min	Standard alert
Estimate-to-final-bill variance (manual, from admin data, not automated telemetry)	Any single order beyond $\pm 15\%$	Reviewed weekly per PRD's counter-metric — this is a business signal as much as a technical one

12. Testing Strategy

12.1 Pyramid

Layer	Tool	Coverage focus
Unit	Vitest	Calculation engine (§6), zone resolution, rate resolution, Zod schemas
Integration	Vitest + a test Postgres instance	API routes against a real (test) database — enquiry creation, OTP flow, admin CRUD
End-to-end	Playwright	Full user journeys on a real browser, mobile viewport
Manual/UAT	—	The PRD §14 launch gate: 8–10 real customers at the counter

12.2 What must have unit tests before merge (non-negotiable)

- The full §6.6 calculator regression suite, including the PRD §7.9 worked example and every edge case in §6.5

- Zone resolution (`resolveZone`) — PIN in zone, PIN in no zone, PIN in overlapping zone definitions (should not happen, but the resolver must have deterministic behaviour if it does)
- Rate resolution (`resolveRate`) — retail vs dealer, missing dealer rate, no variant
- OTP challenge lifecycle — expiry, max attempts, rate limits

12.3 E2E smoke suite (Playwright, mobile viewport, runs nightly against staging)

1. Home → Door A (browse) → filter by colour → open a product → calculate for a room → see an estimate with a non-zero total
2. Home → Door B (wizard) → complete all 4 steps → see a shortlist with box counts
3. From an estimate → submit an enquiry → OTP flow → see confirmation with a reference number
4. Admin login → add a product with one photo → verify it appears in the public catalogue within the ISR revalidation window
5. Admin → edit a rate inline → verify the change reflects on the product page
6. Customer order lookup by phone + OTP → see order list and expand the tracker

A failure in any of these six blocks a production promote (§10.3) — they cover exactly the PRD's launch-definition-of-done journeys.

12.4 Performance testing

Lighthouse CI (or WebPageTest) run against staging on every PR that touches `app/(catalogue)` or `app/(product)` routes, simulated Moto G4 / 4G profile, gated on the PRD N-1.1 LCP < 2.5s budget. A regression here fails the PR the same way a failing unit test would.

12.5 Test data

Staging is seeded with a representative slice of the real catalogue (not synthetic placeholder data) so visual/performance testing reflects real image sizes and real content density. A separate anonymised fixture set (fake names/phones) is used for enquiry/order flow tests, never real customer PII in any non-production environment.

13. Security & Compliance

13.1 Data protection (India's DPDP Act, 2023)

Requirement	Implementation
Consent before collecting personal data	Explicit checkbox/notice before phone number submission on every form (PRD N-4.6), plain-language, not buried in a linked policy
Purpose limitation	Phone/name/address collected only for order fulfilment and enquiry follow-up — never sold or shared with third parties, stated plainly in the privacy policy
Right to erasure	Admin-triggerable customer data deletion; <code>deleted_at</code> soft-delete plus a scheduled hard-purge job after the retention window (§13.3)
Data localisation	Managed Postgres and image CDN both offer India-region hosting — use it; no requirement here to use it, but no reason not to, and it reduces latency too
Breach notification readiness	Sentry + structured logs (§11.2) give the audit trail needed to scope a breach quickly if one ever occurs

13.2 Transport & storage security

Control	Detail
Transport	HTTPS everywhere (Vercel default), HSTS enabled
Encryption at rest	Managed Postgres provider's default at-rest encryption; confirm it's on (it is, by default, on Supabase/Neon)
Secrets	Never in the repo; environment variables in Vercel's encrypted secret store
Admin access	2FA on the Vercel and database provider dashboards themselves — the infrastructure accounts, not just the app

Control	Detail
Dependency security	Automated dependency scanning (Dependabot/Snyk) on every PR

13.3 Retention policy

Data	Retention
Estimates (anonymous)	30 days active (PRD F-5.7), then soft-deleted; hard-purged after 90 days total
Enquiries	Retained 24 months for business/tax record purposes, then reviewed for purge
Orders	Retained per GST/tax record requirements (typically 6–8 years under Indian tax law — confirm with the accountant, this is a legal not engineering decision)
OTP challenges	Purged after 24 hours regardless of outcome
Sentry error logs	Provider default (usually 90 days), never contains raw phone numbers (§11.2)

13.4 Backup & disaster recovery

Control	Detail
Backup frequency	Daily automated snapshot + continuous point-in-time recovery (both Supabase and Neon support this on paid tiers — budget for it, it is not optional)
Backup testing	Quarterly restore drill to a scratch instance — an untested backup is not a backup
RPO (recovery point objective)	≤ 24 hours (PITR makes this closer to minutes in practice)
RTO (recovery time objective)	≤ 4 hours to restore service from a full loss
Image CDN	Provider-managed redundancy; originals also retrievable via the <code>product_image.cdn_public_id</code> reference, so a CDN account issue doesn't silently orphan the catalogue's source images

13.5 GST & business compliance

GST registration number and legal business details in the site footer (PRD N-4.7) are a content requirement, but the **GST rate itself lives in setting , not code** (§4), so a rate change (18% → whatever it becomes) is an admin edit, not a deploy.

14. Technical Risks & Mitigations

#	Risk	Impact	Mitigation
T1	Calculator produces a wrong estimate due to an untested edge case	Fatal to trust — this is the PRD's #1 counter-metric	Mandatory regression suite (§6.6, §12.2) blocks merge; the engine is a pure function isolated from I/O, making it exhaustively testable
T2	Client submits a stale/tampered rate	Financial — wrong price honoured	<code>POST /calculate</code> always re-fetches rates server-side; client-supplied prices are never trusted (§6.5)
T3	Image CDN outage degrades the whole catalogue	High — the photo grid is the product	CDN failure handling (§9.7): cached edge URLs, graceful degradation, never a blocking dependency on page render
T4	OTP/SMS provider DLT template rejected or delayed	High — blocks launch if discovered late	Start DLT registration in Phase 0 (§9.2), not Phase 1 week 6
T5	Admin flow too slow/complex, owner stops updating catalogue	Fatal (mirrors PRD R1)	Single-field PATCH endpoints (§5.4), three-slot enforced photo upload (§7.4), inline rate edit — every admin interaction is designed around the 2-minute target
T6	Database connection exhaustion under a traffic spike (e.g. a local news mention)	Medium	Managed Postgres connection pooling (PgBouncer, usually built into Supabase/Neon), Next.js API routes reuse a pooled Prisma client, not a new connection per request
T7	A rate change mid-session shows the customer one price and bills another	High (mirrors PRD R2)	<code>estimate_line.rate_applied</code> snapshot (§4.1) — the estimate never silently recomputes from a live rate after creation

#	Risk	Impact	Mitigation
T8	Fake/simulated UI elements (live viewer counts, activity toasts) reviewed in the UI concept get shipped as literal hardcoded randomness	High — reputational (mirrors PRD R10)	If any "live" social-proof element ships, it must be backed by a real query (real page-view count, real recent-order count) — flagged here as a build-time gate, not just a design note
T9	Single point of failure: one Vercel account, one DB provider account, no documented recovery runbook	Medium	Backup/DR drills (§13.4) and a one-page "if the site is down" runbook, written before launch, not after the first incident
T10	Scope creep into Phase 3 AI features before Phase 1/2 metrics justify it	Medium — budget and focus risk	The PRD's phase-gate discipline (§14) is a technical decision too: no visualiser code before the catalogue and calculator are proven in production

14.1 Technical debt to track explicitly

- Offset-based pagination (§5.5) is acceptable below ~500 products; flagged for cursor-based migration once the catalogue grows past that.
- `wa.me` deep links (§9.1) have no delivery confirmation; flagged for WhatsApp Business Cloud API migration in Phase 2, not a permanent architecture.
- Colour auto-tagging (§7.3) uses simple k-means; if it proves inaccurate in practice, the PRD's Tier 1 AI roadmap already treats this as a low-risk, revisitable component, not a load-bearing one.

14.2 Document control

Field	Value
Version	1.0 (Draft)
Companion document	Roman Marbel — E-Commerce PRD v1.0
Status	For engineering review

Field	Value
Owner decisions still needed	See PRD §15.3 (Q1-Q12) — several directly affect this document's zone/delivery/GST configuration in setting and zone